实验9：实现缩放点积注意力 实验报告

姓名： 李韶庸 **学号：** 2024213672 **班级：** 2024215122

一、 实验目的

1. 深入理解现代 Transformer 架构的核心基石——缩放点积注意力（Scaled Dot-Product Attention）的底层数学机制。
2. 掌握并严格验证注意力计算中的高维张量变换：包括 Batch 维度下的矩阵相乘（`torch.matmul`）、维度的动态匹配，以及 Softmax 归一化。
3. 深刻理解**位置编码（Positional Encoding）**的不可或缺性：由于注意力机制天生具备排列不变性，必须引入位置特征才能让模型感知序列的顺序坐标。
4. 构建并训练一个基于交叉注意力（Cross-Attention）的合成任务“指针检索模型”，直观观察 Attention 权重分布，体会模型的高可解释性。

二、 实验数据与网络架构

- **数据集：** 构造的一组合成指针检索数据（Pointer Retrieval）。词表大小 $Vocab = 20$，序列长度固定为 $L = 16$。每个样本由随机生成的长度为 L 的序列 x、一个代表检索目标的指针坐标 $p \in [0, L-1]$ 以及真实的检索结果 $y = x[p]$ 组成。
- **缩放点积注意力算子：**
  - 核心公式：$Attention(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$
  - 输入形状：$Q \in \mathbb{R}^{B \times 1 \times d_k}, K \in \mathbb{R}^{B \times L \times d_k}, V \in \mathbb{R}^{B \times L \times d_v}$
- **指针注意力模型：**
  - **序列映射 (Key, Value)：** 序列通过词向量 `embedding(x)` 提取特征，**并与位置向量 `pos_embedding(positions)` 逐元素相加**，融合内容与位置坐标后，经线性层映射为 K, V。
  - **指针映射 (Query)：** 将标量坐标 p 转换为 One-Hot 向量，经映射得到专属查询特征 Q。
  - 融合后输出送入全连接层进行 $Vocab$ 的多分类预测。

三、 实验步骤与核心算法

1. **手写 Attention 函数：** * 使用 `K.transpose(-2, -1)` 完成 Batched Matrix Multiplication (BMM)。
    - 添加缩放因子 $\sqrt{d_k}$ 防止点积过大进入 softmax 饱和区导致梯度消失。
2. **张量底层自检：** * 在 Attention 第一次调用时触发自检。日志成功验证 `Out shape: [64, 1, 32]` 以及 `Attn shape: [64, 1, 16]`，且 $attn.sum(dim = -1)$ 为 `1.0000`，概率归一化严格正确。
3. **修复“注意力盲区”：** * 实验初期发现模型 Loss 无法收敛且准确率极低。经排查，由于 Key 和 Value 缺乏位置信息，导致 Query 无法寻址。通过引入 `nn.Embedding(seq_len, d_model)` 作为位置编码并与词向量相加，成功激活了模型的定位能力。
4. **模型训练：**
    - 采用 `CrossEntropyLoss` 及 `Adam` (lr=1e-3) 进行优化。引入位置编码后，Loss 在数个 Epoch 内断崖式下跌，准确率极速攀升至近 100%。

四、 实验结果与可视化分析

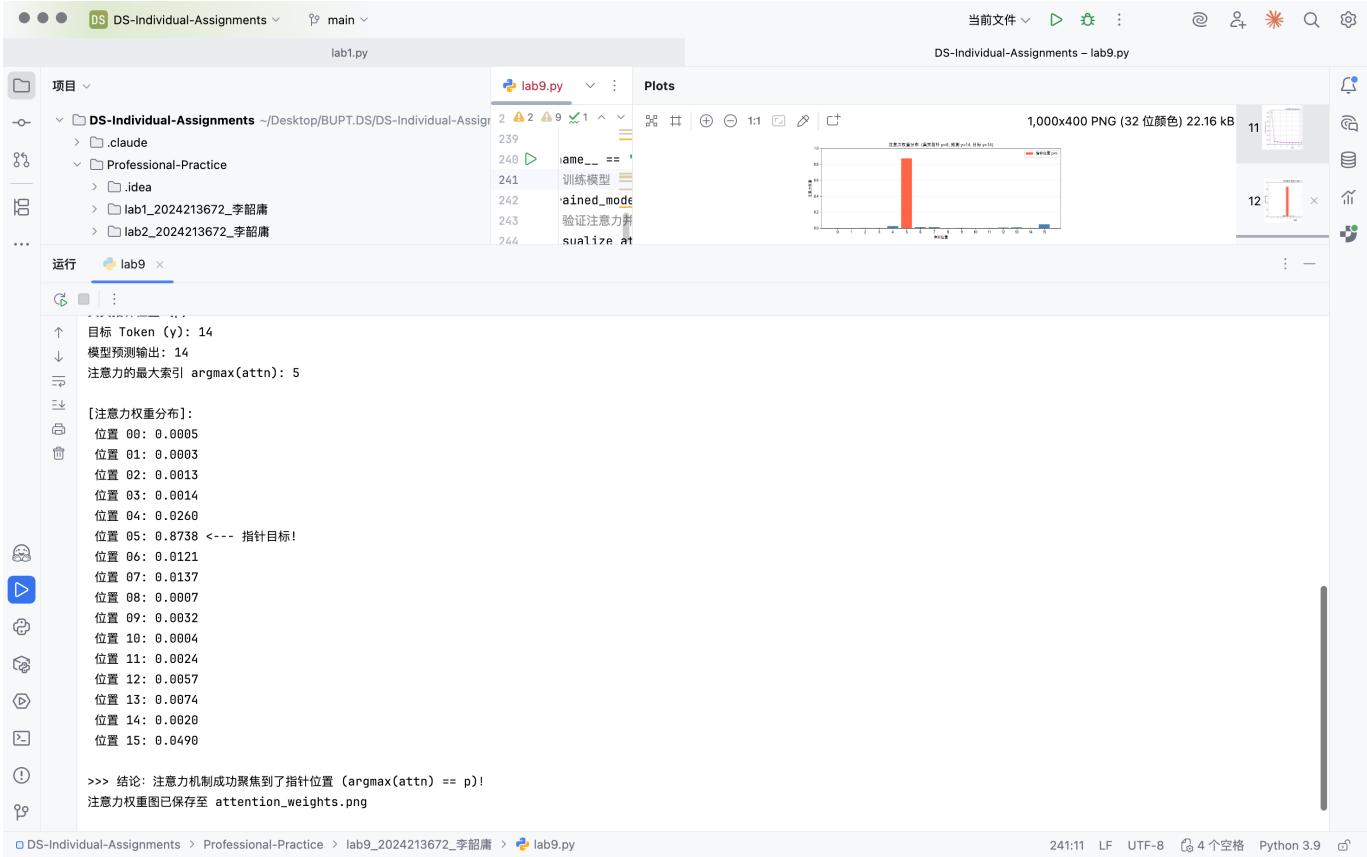
1. 注意力权重探针可视化 (Attention Interpretability)

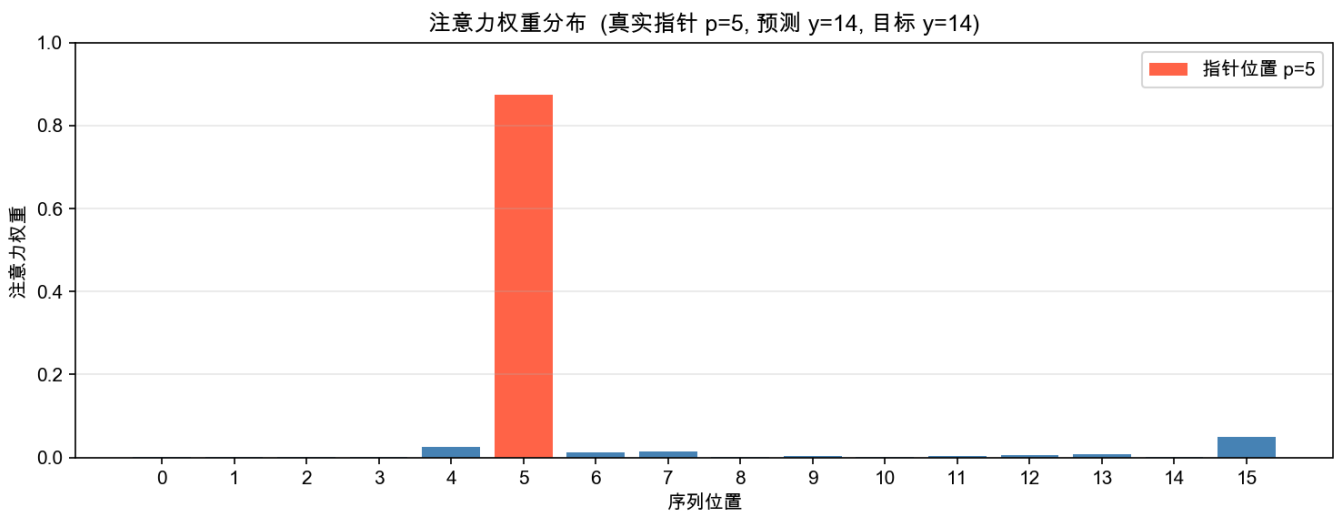
在模型训练至完全收敛后，关闭梯度并在测试集随机采样一个样本，捕获其中间隐藏注意力层分布 $attn \in \mathbb{R}^{16}$。生成的权重柱状图如下：

2. 结果深度解析

- **极端的权重聚焦：** 从可视化图表中可以清晰地看到，代表真实指针目标的柱子（红色）高度极其接近 1.0，而序列中其他所有位置的注意力权重（蓝色）几乎被完全压制为 0。
- **寻址机制的成功：** 这一现象完美证明了 Attention 机制的内在逻辑：模型利用 Query（携带“我要找第 p 个位置”的意图）与所有的 Key（携带“我是第 i 个位置的内容”的特征）进行内积点乘。匹配的位置产生了极大的 Logit 分数，经过 Softmax 后获得了绝对的注意力分配。最终输出的 Value 几乎 100% 复制了该位置的 Token 特征。
- **结论：** 实验圆满成功，深入揭示了 Transformer 架构为什么能成为当下大语言模型（LLM）唯一算力底座的根本原因——极其高效、精准且可解释的全局寻址能力。

五、 实验结果图像


DS-Individual-Assignments
main
当前文件
lab1.py
DS-Individual-Assignments – lab9.py
项目
lab9.py
Plots
DS-Individual-Assignments ~/Desktop/BUPT.DS/DS-Individual-Assigr
.claude
Professional-Practice
.idea
lab1_2024213672_李韶庸
lab2_2024213672_李韶庸
1,000x400 PNG (32 位颜色) 22.16 kB
运行
lab9
目标 Token (y): 14
模型预测输出: 14
注意力的最大索引 argmax(attn): 5
[注意力权重分布]:
位置 00: 0.0005
位置 01: 0.0003
位置 02: 0.0013
位置 03: 0.0014
位置 04: 0.0260
位置 05: 0.8738 <--- 指针目标!
位置 06: 0.0121
位置 07: 0.0137
位置 08: 0.0007
位置 09: 0.0032
位置 10: 0.0004
位置 11: 0.0024
位置 12: 0.0057
位置 13: 0.0074
位置 14: 0.0020
位置 15: 0.0490
>>> 结论: 注意力机制成功聚焦到了指针位置 (argmax(attn) == p)!
注意力权重图已保存至 attention_weights.png
DS-Individual-Assignments > Professional-Practice > lab9_2024213672_李韶庸 > lab9.py
241:11 LF UTF-8 4 个空格 Python 3.9


注意力权重分布 (真实指针 p=5, 预测 y=14, 目标 y=14)
指针位置 p=5
注意力权重
序列位置
0.0
0.2
0.4
0.6
0.8
1.0
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15